PwC × AGENTIC AI

CAPSTONE PROJECT SUBMISSION GUIDELINES

Complete guide for learners — notebook, documentation & grading rubric

100 pts Total Points	**+10 pts** Bonus Points
13 Rubric Criteria	**2** Deliverables

Part 1: Submission Overview

1.1 What You Need to Submit

Your final submission must include two deliverables, packaged in a single folder:

#	Deliverable	Details
01	**Solution Notebook (.ipynb)**	Named as YourFirstName_LastName_ProjectName.ipynbExample: Tom_Holland_AgenticAI.ipynb
02	**Google Document**	Project documentation in the prescribed format (see Part 3). Share with 'Anyone with the link can view' access.

1.2 Notebook Header — Mandatory Markdown Cell

Your notebook must begin with a markdown cell containing all of the following fields exactly as shown:

```
# Capstone Project Submission
**Full Name:** <Your Full Name>
**Uplevel Email:** <yourname@uplevel.edu>
**Problem Statement:** <Name of the Problem Statement>
**Submission Date:** <DD Month YYYY>
```

1.3 Notebook Quality Checklist

Before submitting, ensure your notebook meets all of the following quality standards:

- Use markdown cells between code sections to explain your approach, design decisions, and reasoning.
- Comment your code inline — every function, class, and non-obvious logic block must have a descriptive comment.
- All cells must run from top to bottom without errors. Restart the kernel and run all cells before saving.
- Use meaningful, descriptive variable and function names. Avoid single-letter names outside of loop indices.
- Save the notebook with all cell outputs visible. Graders will not re-run your notebook.
- Organise your notebook with logical sections: data ingestion, processing, agent setup, orchestration, evaluation, UI.
- Ensure all import statements are at the top of the notebook and all dependencies are documented.

Note: Students are not allowed to post their solution notebooks on GitHub, public repositories, forums, or any other public platform. These are graded activities — sharing solutions may result in plagiarism and disqualification.

Allowed: You are encouraged to share your project completion certificate and a write-up of your work (without source code) on LinkedIn or other social channels to showcase your learning journey.

Part 2: Grading Rubric

Your project is evaluated across 13 criteria for a maximum of 100 points, with an optional bonus of up to 10 additional points. Review each criterion carefully and aim for the Excellent band on every dimension.

Excellent (100%) — Target performance	Good (90%)	Satisfactory (75%) | Needs Improvement (40%)

Criteria	Max Pts	Excellent (100%)	Good (90%)	Satisfactory (75%)	Needs Improvement (40%)
Code Readability & Organization	**10**	Single-responsibility functions, consistent naming, logical cell sequencing. Markdown docs sufficient for onboarding without a walkthrough.	Mostly clean with minor naming/structure inconsistencies.	Readable but inconsistent. Minimal documentation.	Unstructured, no comments, hard to follow.
Code Reusability & Modularity	**10**	Well-timed abstractions. Shared utilities (API wrappers, parsers, agent patterns) factored out. Zero copy-paste across components.	Mostly reusable, minor redundancy.	Some reuse, noticeable duplication.	Repeated code throughout, no abstraction.
Error Handling & Failure Recovery	**10**	Closed-loop recovery: detect → diagnose → select strategy (backoff, fallback, degrade gracefully) → log outcome. Covers API limits, malformed docs, LLM refusals.	Retry logic present but open-loop. Logging lacks context.	Some try-except, no recovery loops. Failures crash or return empty.	Minimal handling, frequent crashes.
Document Processing & Text Extraction	**10**	Full lifecycle: ingest, clean, chunk, tag metadata. Chunk size/overlap tuned for retrieval quality. Handles malformed/empty docs without silent failure.	Parses correctly, chunking works but untuned.	Basic parsing, fixed chunks, no overlap, fails on edge cases.	Parsing breaks frequently. No chunking.
Named Entity Recognition & Domain Extraction	**10**	Domain-specific entities beyond generic NER. Typed, deduplicated, linked to source spans.	Standard NER works. Some domain extraction, missing key types.	Generic NER only (person/org/date). No domain specificity.	NER absent or mostly incorrect.
Vector Store & Retrieval Grounding	**10**	Appropriate embedding model + similarity metric. Metadata-filtered search. LLM claims traceable to source chunks — no ungrounded assertions.	Search mostly relevant. Grounding not enforced systematically.	Basic setup, inconsistent search. LLM sometimes asserts beyond retrieved context.	Misconfigured or non-functional. Generation ungrounded.
Agent Building & Integration	**10**	Clear mandate per agent, Pydantic	Good prompts and output structure.	Generic prompts, hardcoded tools, no	Agents incomplete or unreliable.

Criteria	Max Pts	Excellent (100%)	Good (90%)	Satisfactory (75%)	Needs Improvement (40%)
		output contracts, domain-expert prompts. Tool selection intent-driven, not hardcoded. Confidence quantified and propagated.	Some planning but not systematic.	decomposition, no memory.	
Multi-Agent Orchestration	**10**	Agents composed, not just sequenced. Coordination pattern (parallel, consensus, delegation) fits the problem. Conflicts reconciled with defensible strategy.	Coordination works. Consensus present but weighting unjustified.	Agents independent, no real coordination. Simple averaging.	Non-functional or unresolved contradictions.
Workflow & State Management	**10**	Conditional routing on content/risk/confidence. Human-in-the-loop where stakes warrant. Fallback paths for degraded operation. State transitions explicit and auditable.	Main-path routing works. Limited HITL. Basic retries.	Linear pipeline, no routing, no HITL, no fallbacks.	No workflow structure. Ad-hoc agent calls.
Observability & Cost-Latency Design	**10**	End-to-end Langfuse traces with span hierarchy. Cost-latency tradeoffs deliberate: model tiering (GPT-4 vs mini), caching policy, batch strategy — with rationale.	Tracing works. Cost tracked but model selection uniform.	Basic traces, no scoring, single model everywhere, no cost rationale.	No observability. Black-box agents. No cost awareness.
Accuracy & Trustworthiness	**10**	Zero hallucinated facts. Mitigation by design: grounding enforced, schemas constrain generation, confidence gates low-certainty outputs. Eval dataset validates quality.	Some guardrails, no eval dataset. Occasional inaccuracies.	Manual testing only. No systematic mitigation. Some inaccuracies.	Frequent hallucinations, no grounding, no evaluation.
Deployment & Scalability	**5**	FastAPI endpoints tested. Config externalized (dotenv/secrets). Batch + cache reduce redundant LLM calls. Handles realistic workloads.	Endpoints work. Mostly clean config. Gaps in caching/batch.	Not production-ready. Hardcoded config.	Notebook-only. Manual cell execution.
UI & Usability	**5**	Gradio interface with file upload, tabbed results, embedded visualizations. Non-technical stakeholder can use without tutorial.	Functional UI, some views/visualizations missing.	Basic single-view interface. No visualizations.	Confusing or missing UI.

BONUS — Unique Enhancements / Additional Features (+10 points)

A substantive feature built beyond the core requirements. Examples include: knowledge graphs, predictive analytics, custom eval frameworks, novel coordination patterns, or domain innovations showing independent judgment. Minor UI tweaks or formatting changes do not qualify for bonus marks.

Part 3: Google Document — Format & Template

Create a Google Document using the structure below. This document demonstrates your understanding of the project and is designed to be shareable on LinkedIn as a professional portfolio piece. Each section maps directly to what graders look for during evaluation.

Cover Section

Required information at the top of your document *required*

- Project title
- Your full name and Uplevel email address
- Problem statement name
- Submission date

Document Sections

1 — Problem Statement & Objective *required*

- What business or domain problem does this project address?
- Who is the end user or key stakeholder?
- What does a successful outcome look like?

2 — System Architecture & Design *required*

- High-level architecture diagram (screenshot or embed from your notebook)
- Agent roles — the mandate and responsibilities of each agent
- Orchestration pattern chosen (parallel, sequential, consensus, delegation) and justification
- Workflow and state management approach

3 — Technical Implementation *required*

- Document processing pipeline: ingestion, chunking strategy, metadata tagging
- Embedding model and vector store configuration — similarity metric and top-k rationale
- NER approach — domain-specific entities extracted and how they were typed/linked
- Prompt engineering strategy per agent
- Error handling and recovery loops implemented

4 — Observability & Cost Design *required*

- Langfuse trace setup — span hierarchy and what was instrumented
- Model tiering decisions (e.g., GPT-4o for critical steps, mini for routing/classification)
- Caching and batching strategies implemented
- Cost-latency tradeoff rationale and documented decisions

5 **Evaluation & Accuracy** *required*

- How did you evaluate output quality? (golden pairs, annotated examples, spot-checks)
- Hallucination mitigation strategy — grounding enforcement and confidence gates used
- Key accuracy results, observations, or limitations found

6 **Deployment & UI** *required*

- FastAPI endpoint design and how it was tested
- Secrets and configuration externalization approach (dotenv, environment variables)
- Gradio/ Hugging face/ Fasat API/ AWS/Vercel interface features: file upload, tabbed results, embedded visualizations
- Screenshot of the working user interface

7 **Challenges & Learnings** *recommended*

- Top 2–3 challenges encountered during the project and how you resolved them
- What you would do differently given more time or resources
- Key technical or domain insights gained throughout the project

8 **Unique Enhancements (Bonus)** *optional*

- Describe any feature built beyond the core requirements
- Show measurable improvement or novel capability it introduces
- Examples: knowledge graph, custom eval framework, novel agent coordination pattern

Part 4: How to Submit

4.1 Prepare Your Submission Folder

Package all files in a single folder before uploading. Your folder structure should look like this:

```
YourName_ProjectName/
    ├── YourFirstName_LastName_ProjectName.ipynb
    ├── [any supporting datasets or config files]
    └── [screenshots or additional assets if applicable]
```

Note: The Google Doc link is submitted separately in the Google Form — it does not go inside the folder.

4.2 Pre-Submission Checklist

Go through every item before uploading:

- Restart kernel → Run All → verify no errors before saving the notebook.
- The mandatory markdown header cell is present at the very top of your notebook with all required fields filled in.
- All cell outputs are visible and saved — graders will not re-run your notebook.
- Your notebook file is named correctly: FirstName_LastName_ProjectName.ipynb
- Code is well-commented and markdown cells explain each major section of the notebook.
- Your Google Doc is shared with 'Anyone with the link can view' access and covers all 8 prescribed sections.
- Your submission folder is zipped and ready to upload.

4.3 What to Include in the Google Form

When filling out the Google Form, you will be asked to provide:

1. Your full name and Uplevel email address
2. Name of the problem statement you solved
3. Upload of your zipped submission folder
4. Shareable link to your Google Document

Academic Integrity: Do not share your solution notebook (.ipynb) or source code on GitHub, public forums, Kaggle, or any other public platform. These are graded activities. Sharing solutions constitutes plagiarism and may result in disqualification from the program.

Share Your Achievement: You are welcome and encouraged to post your project completion certificate and a write-up of what you built (without code) on LinkedIn or other social media channels.